Beyond %Resolved:Reassessing SWE-bench Through Human and LLM Code Review

Myeongmuk Kang and Geunseok Yang
Department of Computer Science and Engineering, Hankyong National University
Anseong, Republic of Korea
Email: {myoungmuk819, gsyang}@hknu.ac.kr
ORCID: 0009-0002-5351-0269 (Kang); 0000-0001-5677-5129 (Yang)

***Abstract*—Passing benchmark tests does not establish that a maintainer would merge a coding-agent patch. Large language model (LLM) reviewers offer a scalable additional assessment, but their decisions also require validation. We study test-passing patches from four coding-agent scaffolds on SWE-bench Lite and Verified, pairing agent patches with developer-merged gold patches under source-blinded review. The headline naive reviewer, Qwen2.5-Coder-14B-AWQ, rejects 94.79% and 93.3% of the gold patches on Lite and Verified. After jointly adjusting test-context disclosure and the merge-blocking rubric, its gold rejection falls to 27.08% and 25.26%; the same data were used during rubric development and evaluation, so this is not independent calibration validation. Across ten open-weight reviewer/prompt configurations on Verified, agent and gold acceptance rates correlate at $r = 0.994$, while agreement across five adjusted reviewers is low (Fleiss $\kappa \approx 0.02$). A hosted Claude reviewer accepts $164/194$ agent patches and $191/194$ paired gold patches (exact McNemar $p < 0.001$), but its exact model snapshot was not recorded and these decisions were not validated against maintainers. The archived failing-patch comparison also supplied an incorrect test-pass assertion and used different tasks; it cannot establish discrimination under correct test information. A single non-independent rater's 40-packet pilot provides descriptive observations only. Our contribution is a control-based diagnosis of reviewer-dependent acceptance, together with packets, verdicts, and analysis procedures. Historical gold acceptance is a useful reference, but neither agreement with that reference nor a reviewer-specific agent–gold difference establishes real-world mergeability.**

I. Introduction

The dominant way to evaluate large language model (LLM) coding agents is execution-based: SWE-bench [1] frames the task as "GitHub issue → patch" and declares a task *resolved* when the generated patch makes a held-out `FAIL_TO_PASS` test pass while preserving `PASS_TO_PASS` tests. The leaderboard metric %Resolved has become the field's primary yardstick.

As execution scores rise, review-based criteria increasingly inform comparisons of coding agents. LLMs make such assessments scalable, but interpreting a review score requires checking the reviewer as well as the candidate.

Yet in real software development, passing tests is necessary but not sufficient for a change to be accepted: a pull request must also survive *code review*, where maintainers weigh functional completeness, regression risk, minimality, maintainability, project conventions, and test adequacy before merging [2], [3]. That test-passing patches can still be wrong is a long-standing observation in automated program repair, where "plausible but incorrect" patches and test overfitting have been studied for a decade [4]–[7]. For agentic software engineering the concern is broader: the question is not only whether a multi-file, agent-generated patch is *correct* but whether it is *mergeable*.

A natural research program follows: complement %Resolved with a review-based criterion and quantify the gap between them. Because human review at benchmark scale is expensive, the obvious instrument is an LLM reviewer, mirroring the now-common LLM-as-judge paradigm [8]. This paper asks whether that instrument actually works.

We operationalize review-based evaluation on test-passing Agentless [9] patches from SWE-bench Lite and Verified [1], [10]. Each agent patch and its developer-merged gold counterpart is rendered as a source-blinded review packet. Gold acceptance measures agreement with a known historical merge outcome under those packet conditions. It does not establish that a gold patch is defect-free or that a new reviewer's objection is false. Our measured outcome is *reviewer-specific acceptance*, which we distinguish from an original maintainer's merge decision throughout the paper.

For Qwen2.5-Coder-14B-AWQ, the naive prompt rejects 94.79%/93.3% of gold patches on Lite/Verified. Jointly adjusting test-context disclosure and the blocking criterion increases both agent and gold acceptance. Because the changes were not ablated and development inspected the same gold cohort, we report a configuration comparison and subgroup stability, not independent calibration generalization or an identified cause.

The open reviewers show an empirical *validity–discrimination tension*: configurations with high historical-gold agreement can also accept most agent patches, while more rejecting configurations can disagree with historical gold acceptance. Across five models and two prompts, aggregate agent and gold acceptance correlate at $r = 0.994$. This correlation does not identify a per-patch decision mechanism. A hosted Claude reviewer provides a different operating point: 84.5% agent acceptance versus 98.5% gold acceptance on 194 paired tasks ($p < 0.001$). That difference is specific to the archived reviewer and rubric; the exact hosted model ID was not logged, and maintainer agreement remains

Corresponding author: Geunseok Yang (gsyang@hknu.ac.kr).

unvalidated. Moreover, the failing-patch comparison reused a false test-pass assertion, so it supplies no clean confirmation of functional discrimination.

We examine reviewer scale within Qwen2.5-Coder, extend the comparison to Llama and DeepSeek, and compare three adjusted Qwen reviewers on four coding-agent scaffolds. Reviewer choice matters: low inter-reviewer agreement persists, and Qwen-32B detects agent–gold differences for SWE-agent and OpenHands that Qwen-14B does not. A single non-independent human rater adds a limited descriptive comparison, not a population-level human validation.

Our contributions are:

- A paired historical-gold comparison showing how reported review acceptance changes with the reviewer and prompt, including the contrast between the naive and adjusted Qwen configurations (§IV-B).
- A documented diagnosis of limitations in control-based validation: reused calibration tasks, joint prompt changes, different positive/negative task populations, and an incorrect pass assertion in the archived negative-control condition (§III).
- Cross-model and cross-agent results that separate aggregate acceptance, paired differences, and inter-reviewer agreement; these outcomes are not substitutes for maintainer-level validation (§IV-B).
- A replication package containing source-blinded packets, normalized verdicts, exact prompts and schemas, analysis code, and the 40-packet exploratory human pilot.

Control examples and human-guided rubrics have precedents [11], [12]. Our contribution is the SWE-bench-specific historical-acceptance comparison and its empirical diagnosis, not the first use of calibration or positive controls.

II. Background

Execution-based agent evaluation. SWE-bench [1] casts coding-agent evaluation as resolving real GitHub issues and scores an attempt by running the repository's test suite: a task is *resolved* when the agent's patch turns the `FAIL_TO_PASS` tests green without breaking the `PASS_TO_PASS` tests. SWE-bench Verified is a 500-task subset that human annotators confirmed to be well-specified and solvable [10], partly in response to concerns that some original tasks were underspecified or had weak tests. A large ecosystem of scaffolds (Agentless [9], SWE-agent [13], OpenHands [14], AutoCodeRover [15], and the multimodal extension [16], among many others) now competes on %Resolved, which has become the field's de-facto figure of merit. Execution-based scoring also has a known fragility: benchmark contamination of code LLMs can inflate it [17], which further motivates evaluation criteria that do not reduce to a single passed test. Crucially, %Resolved is an *execution* signal: it certifies that a specific, held-out test passes, not that the change is one a project would actually accept. Our work does not compete on the metric; it treats the test-passing patches as the object of study.

Anatomy of a SWE-bench task and why it shapes review. Each SWE-bench task bundles an issue (the natural-language `problem_statement`), a base commit, the developer's merged code change (the *gold* `patch`), and a `test_patch` containing the `FAIL_TO_PASS` test that encodes the bug plus `PASS_TO_PASS` regression tests.

At evaluation time the candidate is assessed using held-out test information supplied by the benchmark harness. Our review packets contain the candidate source diff and issue statement, without the held-out test patch. Thus the reviewer does not see the same material as a full pull request with CI output and discussion. Both agent and gold packets omit the held-out tests. An "insufficient tests" concern may consequently reflect the review format; its frequency alone does not establish whether a particular objection is justified.

III. Our Approach

We first fix public predictions and official resolved labels, then render paired agent/gold review packets, collect structured reviewer decisions, and measure acceptance relative to historical gold outcomes. We also examine archived reviews of harness-failing patches, whose incorrect test-pass instruction is documented explicitly below. Source blinding and shared rendering reduce explicit source cues; they do not eliminate task familiarity, stylistic cues, missing context, or calibration-data reuse.

A. Research questions

- **RQ1.** Under the specified LLM reviewer, what fraction of test-passing agent patches are accepted, and how large is the apparent Resolution–Mergeability gap?
- **RQ2.** How often does the reviewer accept developer-merged gold patches, and how does rubric adjustment change the paired agent–gold comparison?
- **RQ3.** What failure modes do reviewers cite, and how do they shift between naive and adjusted review?
- **RQ4.** How much do acceptance decisions agree across reviewer models of different scale and family?
- **RQ5.** How do paired agent–gold comparisons vary across coding-agent scaffolds and adjusted reviewers?

IV. Experiments

A. Experimental setup

1) Data and test-passing labels: We study both standard SWE-bench splits: Lite (300 tasks) and the human-validated Verified (500 tasks), each task pairing a GitHub issue with the developer's merged fix and a hidden test that encodes the bug. The agent under study is Agentless v1.5 [9], a strong, widely-cited, and, importantly for reproducibility, *publicly released* scaffold whose predictions we obtain directly rather than re-running, eliminating any sampling nondeterminism on our side.

Label provenance. We attribute the reused predictions to the official Agentless GPT-4o submission by archive identity, not by inferring the model from test outcomes. We pin the SWE-bench experiments metadata and result lists to commit `40f164d5b8f1d249bf95a6df8b74b577fd8e519d` and the entry `20241028_agentless-1.5_gpt4o`. The

authors' v1.5.0 release prediction files for Lite and Verified are byte-identical to the official submission files (all 300 and 500 predictions, respectively). All 96 Lite and 194 Verified agent patches in our preserved review packets match these files. Rebuilding the main review packets reproduces all 580 records, keys, and their order. This establishes consistency with the attributed public artifacts; it does not independently observe the provider's internal generation process.

We adopt the official resolved sets, which exactly match the preserved study labels: 96/300 resolved on Lite (32.0%) and 194/500 on Verified (38.8%). Separately, archived runs of the official Docker harness on a historical 12-instance sample agree with the adopted Lite labels in all 12 cases. This sample is a limited execution check, not proof of full prediction identity. We did not rerun the complete 800-task evaluation in the archive verification. The main acceptance comparisons concern patches labeled test-passing by the corresponding official submission report.

Multiple agents. To test whether our conclusions depend on the choice of agent, we additionally study three further scaffolds on Verified, obtained the same way from the public submission archive: a weak retrieval baseline (RAG+GPT-4, %Resolved 2.8), and two strong agentic systems (SWE-agent+Claude-3.5 at 33.6, OpenHands-CodeAct at 53.0). With Agentless this gives four agents of four distinct designs spanning a 19× range in %Resolved, with resolved labels taken from each entry's official harness report. We verify their reviewed patches against official prediction files or per-instance `patch.diff` artifacts and review them with the adjusted reviewers against the same shared gold control. Across the five split–agent entries, all 737 preserved resolved patches match their public sources, and all resolved-ID sets match the pinned official reports. The 302-patch Verified negative-control pool also matches the official Agentless predictions; this is a provenance check on the pool, not a claim that every reviewer evaluated all of it.

2) Blind review packets: For every resolved (test-passing) instance we build two blind, pull-request-like packets: one containing the *agent* patch and one containing the *developer-merged gold* patch. Each packet shows the linked issue (problem statement), repository, base commit, the unified diff, and a changed-file summary. To mask explicit source information, packets never reveal the candidate source, never show the gold patch alongside the agent patch, and never include the held-out `test_patch` or issue hints; packet order is shuffled with a fixed seed. The source label is stored in a separate key file used only at analysis time.

3) Reviewers and calibration: The open-weight panel comprises Qwen2.5-Coder-7B/14B/32B-Instruct (AWQ for 14B/32B), Llama-3.1-8B-Instruct, and DeepSeek-Coder-6.7B-Instruct [18]. We use family-qualified names Qwen-7B, Qwen-14B, Qwen-32B, Llama-8B, and DeepSeek-6.7B. Coding-agent scaffolds generate the candidate patches; these reviewers perform single-pass judgment with a fixed JSON schema. They are not the same systems or tasks. The panel provides a scale comparison and two additional families, not a claim to cover the best available review systems.

The schema contains a decision (merge, request changes, or reject), issues with severity and failure-mode tags, and self-reported confidence. A verdict counts as accepted only if the decision is merge and it has no blocking issue. Confidence is not used by any reported metric. The fixed taxonomy names F1 functional incompleteness, F2 regression risk, F3 test overfitting, F4 non-minimal change, F5 maintainability, F6 convention mismatch, F7 insufficient tests, F8 fragility, F9 API/compatibility, F10 performance, and F11 documentation gaps. These are reviewer-reported concerns, not independently adjudicated defects.

Rubric adjustment. The naive prompt asks for rigorous review and already asserts that automated tests pass. The adjusted prompt additionally explains that test files are intentionally absent and narrows blocking severity to concerns over which the reviewer would refuse to merge. Its exact assertions include "reproduction test ... and ... regression test suite have ALREADY been run ... and PASS", "absence of test files ... must NOT by itself be treated as a problem", and "Reserve a 'blocking' severity ONLY for problems that would genuinely make you refuse to merge." The user-prompt heading and final reminder also assert a pass. The replication package preserves the complete system and user templates in `PROMPTS.md`, not only these excerpts.

We call the revised configuration *adjusted*; historical artifacts retain "calibrated" as their original label. Here calibration means manual gold-referenced rubric adjustment, not probability/confidence calibration. According to the preserved development account, the adjustment followed inspection of naive gold verdicts from the same cohort later evaluated; agent verdicts were not inspected. Only the final rubric was preserved, so the number of intermediate revisions and inspected examples is not recoverable. "High and stable" gold acceptance was a qualitative criterion, not a preset numeric threshold. Excluding agent examples does not make rubric development independent of the evaluation tasks, nor exclude a calibration-induced reduction of an agent–gold difference. Both prompt changes were applied together; we cannot identify their separate contributions.

Subset stability. We reuse fixed verdicts, sort instance IDs, and assign index modulo five to five groups. No rubric is developed with a group held out. For example, Qwen-14B gold rejection is $25.3 \pm 5.5\%$ across these groups and the hosted reviewer is $1.6 \pm 1.3\%$ (mean and population standard deviation of group percentages). This describes within-cohort variation; it is not cross-validation, does not rule out adaptation to gold examples, and provides no unseen-task calibration guarantee.

Serving and retries. Open models are served locally through an OpenAI-compatible endpoint. The first JSON request uses temperature 0.0 and a 1,200-token output limit. The script permits three total attempts; subsequent attempts use temperature 0.3 after errors or unparseable output. The final valid normalized verdict is retained; remaining parse failures are excluded and counted. This differs from uniformly greedy decoding. For DeepSeek on Verified, valid naive

agent/gold counts are $190/191$, adjusted counts are $191/192$, and $296/302$ failing-patch outputs are valid. Other main configurations have complete $194/194$ agent/gold outputs on Verified. Fresh inference also depends on weights, serving software, hardware, and numerical execution; temperature zero does not ensure bitwise reproducibility.

Hosted reviewer identity. The additional frontier comparison uses Claude Code CLI in headless mode, with the same assembled rubric and a JSON-only instruction. The archived command is `claude -p ... -output-format text`, without an explicit `-model`. Neither an exact model ID/snapshot nor the CLI version and provider decoding configuration was recorded in the verdicts. The development account places the runs on June 30–July 1, 2026; this is not a request-level execution log. We report the archived results as "Claude CLI (snapshot unrecorded)", without inferring an ID from current defaults or code-authoring metadata. Thus exact frontier inference reproduction remains unresolved; released verdicts support analysis reproduction only.

4) Metrics: **Rates and reference outcomes.** %Resolved is the official execution-based test-pass percentage. Review Acceptance among Plausible (RAP) is the accepted fraction of resolved agent patches with valid verdicts. Mergeable Resolved Rate (MRR) retains the original name for the observed accepted-and-resolved fraction of all benchmark tasks; neither name establishes real-world mergeability. The apparent Resolution–Mergeability gap is %Resolved minus MRR, in percentage points. Writing R for the officially resolved set, $A_m \subseteq R$ for valid agent verdicts from reviewer m, and N for all tasks,

$$\mathrm{RAP}_m = \frac{\sum_{i \in A_m} m(a_i)}{|A_m|}, \qquad \mathrm{MRR}_m^{obs} = \frac{\sum_{i \in A_m} m(a_i)}{N}.$$

For complete outputs, $\mathrm{MRR} = |R|\mathrm{RAP}/N$. With parse failures, MRR is only the observed accepted fraction; missing outputs are not demonstrated rejections. All rates and paired tests report their applicable denominators.

Gold acceptance measures agreement with historical acceptance, and *gold rejection* is its complement. The archived term "false rejection" uses this historical reference; it does not establish that an individual concern was mistaken. *Gold blocking rate* is the proportion of valid gold verdicts reporting at least one blocking issue. It differs from gold rejection because request-changes and other decisions can lack blocking issues. We do not label these objections hallucinations without adjudicating their factual validity.

Statistics. We compare valid paired agent/gold binary verdicts for the same task using the two-sided exact McNemar test. It conditions on discordant pairs: gold-only acceptance versus agent-only acceptance. Wilson 95% intervals describe proportions. Non-significance is failure to detect a difference, not evidence of equivalence; tests are exploratory and do not establish a universal acceptance threshold. Fleiss' κ uses the common valid packets across a reviewer panel, and pairwise Cohen's κ compares two raters. Low chance-corrected agreement need not mean 50% raw agreement. Parse failures can change both the evaluated population and the shared paired subset.

5) Controls: Historical gold references comprise agent-resolved paired subsets. A separate full-split test-blind 7B control is reported in Section IV-C; it does not revalidate the historical instruments. The archived *negative-control* pool contains 302 non-empty Agentless patches from 306 unresolved Verified instances (four patches were empty). Harness non-resolution can reflect failure to satisfy FAIL_TO_PASS, regression on PASS_TO_PASS, or other recorded harness failures; it alone does not identify an unfixed target behavior.

Incorrect pass information. These failing-patch reviews reused the unchanged adjusted system prompt, user heading, and reminder asserting that tests passed. This assertion was false for the pool. We retain the observed rates as an archived comparison under incorrect pass information, not a clean test of discrimination with accurate test information. The condition was not independently designed in advance as a stress test, and the causal effect of the false assertion was not measured against a corrected condition. Positive and negative tasks also have zero overlap, so their rates are not a matched correctness contrast.

All five open reviewers were attempted on 302 packets (DeepSeek yielded 296 valid verdicts). The hosted run stopped during expansion at 81 completed packets. In the preserved shuffled sequence these are positions 1–79 and 81–82; position 80 is absent and its cause is unrecorded. This is an incomplete execution sample, not an exact first-81 prefix or a prespecified random sample. We report counts and selection limits alongside rates, and withdraw claims that this condition validates normal functional discrimination or identifies a uniquely valid reviewer.

B. Results

RQ1 (agent-only apparent gap). The headline reviewer in this question is Qwen2.5-Coder-14B-AWQ, used as an illustrative anchor rather than an optimal or representative judge. The scale and family panels bound this reviewer-specific result. Its naive prompt accepts 6.25% of resolved Lite patches and 6.19% on Verified. Relative to official %Resolved, the corresponding observed gaps are 30.0 and 36.4 percentage points (Table I). These numbers describe the chosen reviewer, without yet showing that a maintainer would reject those patches.

RQ2 (historical-gold comparison). The same naive Qwen-14B reviewer rejects 94.79% of gold patches on Lite and 93.3% on Verified. After joint rubric adjustment, gold acceptance rises to 72.92%/74.74%, while agent acceptance rises to 70.83%/72.16%. On Lite the latter has a Wilson 95% interval $[61.08, 78.98]$. The adjusted paired comparisons do not detect an agent–gold difference ($p = 0.8238$ Lite, 0.49956 Verified), but only 35 Verified pairs are discordant. Neither non-significance nor shared movement in rates demonstrates equivalent patch quality or an independently validated rubric.

Within the evaluated Qwen family, naive Lite gold rejection rises from 30.21% (7B) to 94.79% (14B) and 100.0% (32B).

TABLE I

REVIEW-BASED ACCEPTANCE UNDER NAIVE VS. ADJUSTED REVIEWERS. RAP AND GOLD-ACCEPTANCE ARE PERCENTAGES; THE NAIVE REVIEWER REJECTS NEARLY ALL MERGED CODE IN THE HEADLINE QWEN-14B SETTING. ADJUSTED QWEN COMPARISONS DO NOT DETECT A PAIRED DIFFERENCE; THIS IS NOT AN EQUIVALENCE RESULT (EXACT MCNEMAR p). EACH ROW HAS n VALID AGENT AND GOLD VERDICTS.

Reviewer	Prompt	Split	n	RAP%	Gold acc.%	Gold rej.%	Paired p
Qwen-7B	Naive	Lite	96	72.92	69.79	30.21	0.5811
Qwen-7B	Naive	Verified	194	73.20	70.62	29.38	0.5224
Qwen-7B	Adjusted	Lite	96	86.46	87.50	12.50	1.0000
Qwen-7B	Adjusted	Verified	194	91.24	87.11	12.89	0.1516
Qwen-14B	Naive	Lite	96	6.25	5.21	94.79	1.0000
Qwen-14B	Naive	Verified	194	6.19	6.70	93.30	1.0000
Qwen-14B	Adjusted	Lite	96	70.83	72.92	27.08	0.8238
Qwen-14B	Adjusted	Verified	194	72.16	74.74	25.26	0.4996
Qwen-32B	Naive	Lite	96	0.00	0.00	100.00	1.0000
Qwen-32B	Naive	Verified	194	0.52	0.52	99.48	1.0000
Qwen-32B	Adjusted	Lite	96	51.04	56.25	43.75	0.4244
Qwen-32B	Adjusted	Verified	194	57.73	61.86	38.14	0.3497

Adjusted rates are 12.5%, 27.08%, and 43.75%, respectively. All adjusted Agentless paired comparisons in this scale ladder have $p > 0.15$; this does not extend to every coding agent or establish that model scale causes stricter judgment.

Additional families. Llama-8B and DeepSeek-6.7B extend the selected panel beyond Qwen (Table II). Their naive gold-rejection rates are 84.02% and 80.63%, falling to 17.53% and 61.98% with adjustment. On Agentless Verified no adjusted open reviewer has a paired $p < 0.05$; Llama's $p = 0.05958$ is close to that descriptive cutoff. The valid DeepSeek counts are incomplete and its paired test uses the common tasks, reported in the table. These configurations cannot establish generalization to all model families or independence from calibration examples.

Observed validity–discrimination tension. The highest gold-accepting adjusted open reviewers also accept many Agentless patches: Qwen-7B accepts 91.2% of agent patches and Llama-8B 89.2%. Qwen-32B and DeepSeek-6.7B reject larger shares of gold as well as agent patches. This motivates checking both historical-gold agreement and decision sensitivity. We use "validity–discrimination" descriptively; there is no prespecified gold-acceptance cutoff, no validated functional-discrimination test in the flawed negative condition, and no proof that leniency causes a paired null.

For all ten open model/prompt configurations on Verified, agent and gold acceptance lie near the diagonal (Fig. 1): their aggregate correlation is $r = 0.994$ and the mean absolute difference is about 3.5 points. An aggregate association across selected configurations does not show that each reviewer uses a single global threshold or lacks instance-level information. It does show that interpreting agent acceptance without the accompanying gold reference can conceal large reviewer/prompt differences.

Archived failing-patch comparison. Table III retains the recorded acceptance rates while exposing both denominators and the incorrect pass condition. For example, Qwen-7B accepts $254/302$ failing candidates under that assertion, versus $169/194$ gold candidates on different tasks. This is a diagnostic reason to inspect instructions and populations before interpreting acceptance. It is not an estimate of the causal effect of test status. The hosted reviewer's $43/81$ rate likewise cannot establish that it alone passes a clean negative control. Correctly informing the reviewer and comparing suitable task populations would be needed to test that claim.

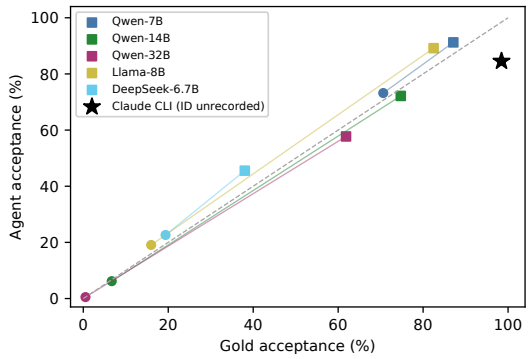


Fig. 1. Verified acceptance for five open reviewers under naive (circles) and adjusted (squares) prompts. Their aggregate agent/gold rates correlate at $r = 0.994$; this does not establish an instance-level decision rule. The archived Claude CLI result (star; snapshot unrecorded) has a larger gold–agent gap in this selected panel.

Hosted reviewer paired comparison. On all 194 Agentless Verified paired tasks, the archived Claude CLI reviewer accepts $164/194$ agent and $191/194$ gold patches. There are 29 gold-only and 2 agent-only accepted pairs (two-sided exact McNemar $p \approx 4.63 \times 10^{-7}$). The paired acceptance difference is 13.9 points, with the archived bootstrap 95% interval $[8.8, 19.6]$. This is a reviewer-specific statistical difference, not demonstrated maintainer agreement or functional correctness. It coexists with high gold acceptance in this sample, but the exact hosted snapshot is unrecorded and no predeclared control threshold was applied. Pairwise agreement with open reviewers ranges from approximately -0.02 to 0.13; agreement with the pilot rater is $\kappa = 0.167$. These comparisons do

TABLE II

CROSS-FAMILY REVIEWERS ON VERIFIED. RATES USE VALID VERDICTS, AND $n_a/n_g/n_p$ GIVES ADJUSTED AGENT/GOLD/PAIRED COUNTS. NON-SIGNIFICANT PAIRED COMPARISONS DO NOT ESTABLISH EQUIVALENCE.

Reviewer	Naive gold rej.%	Adj. gold rej.%	Adj. RAP%	Adj. gold acc.%	$n_a/n_g/n_p$	Paired p
Qwen-7B	29.38	12.89	91.24	87.11	194/194/194	0.1516
Qwen-14B	93.30	25.26	72.16	74.74	194/194/194	0.4996
Qwen-32B	99.48	38.14	57.73	61.86	194/194/194	0.3497
Llama-8B	84.02	17.53	89.18	82.47	194/194/194	0.0596
DeepSeek-6.7B	80.63	61.98	45.55	38.02	191/192/189	0.1174

TABLE III

ARCHIVED ADJUSTED REVIEWS ON DIFFERENT GOLD AND FAILING TASK SETS. FAILING-PATCH PROMPTS INCORRECTLY ASSERTED TEST PASSAGE. THESE RATES DO NOT VALIDATE DISCRIMINATION UNDER CORRECT TEST INFORMATION. k/n DENOTES ACCEPTED/VALID VERDICTS.

Reviewer	Gold k/n	Gold%	Failing k/n	Failing%
Qwen-7B	169/194	87.1	254/302	84.1
Qwen-14B	145/194	74.7	200/302	66.2
Qwen-32B	120/194	61.9	120/302	39.7
Llama-8B	160/194	82.5	260/302	86.1
DeepSeek-6.7B	73/192	38.0	133/296	44.9
Claude CLI	191/194	98.5	43/81	53.1

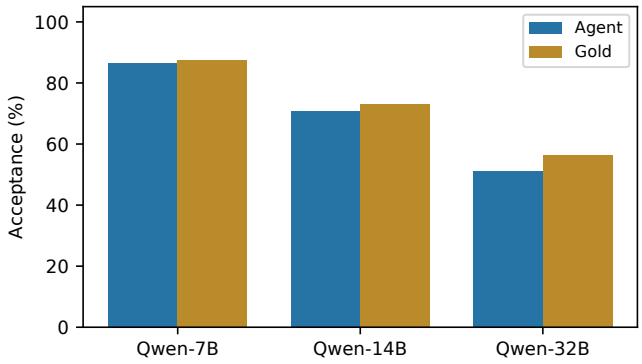


Fig. 2. Adjusted Qwen acceptance rates on Lite. Agent and gold rates are close at each reviewer size, while absolute acceptance varies by reviewer. Paired tests do not detect differences here; non-significance is not equivalence.

TABLE IV

DECISION DISTRIBUTION UNDER THE ADJUSTED REVIEWERS. NON-MERGE IS DOMINATED BY REQUEST-CHANGES RATHER THAN REJECT, AND AGENT AND GOLD DISTRIBUTIONS TRACK EACH OTHER.

Reviewer	Source	Split	Merge%	Changes%	Reject%
Qwen-7B	agent	Lite	87.5	10.4	2.1
Qwen-7B	gold	Lite	89.6	9.4	1.0
Qwen-7B	agent	Verified	92.3	5.7	2.1
Qwen-7B	gold	Verified	88.7	7.7	3.6
Qwen-14B	agent	Lite	70.8	26.0	3.1
Qwen-14B	gold	Lite	72.9	24.0	3.1
Qwen-14B	agent	Verified	72.2	24.7	3.1
Qwen-14B	gold	Verified	74.7	22.2	3.1
Qwen-32B	agent	Lite	51.0	46.9	2.1
Qwen-32B	gold	Lite	56.2	42.7	1.0
Qwen-32B	agent	Verified	57.7	41.8	0.5
Qwen-32B	gold	Verified	61.9	37.6	0.5

TABLE V

FAILURE MODES CITED ON NON-ACCEPTED AGENT PATCHES UNDER THE ADJUSTED REVIEWER (COUNT OF PATCHES FLAGGED, ANY SEVERITY).

Code	Cited concern	Lite	Verified
F1	Functional incompleteness	9	7
F2	Regression risk	6	18
F4	Non-minimal change	5	11
F5	Maintainability	3	8
F6	Convention mismatch	11	11
F7	Insufficient tests	1	0
F8	Fragile approach	4	12
F9	API/compatibility	1	3
F10	Performance	1	2

not validate the reviewer as a maintainer proxy, and we make no recommendation to use it for merge decisions.

Figure 2 shows the adjusted Qwen acceptance rates on Lite. Agent and gold rates are close, while their absolute levels differ substantially by reviewer. Table IV reports the schema's decision categories: most non-merge actions are request-changes rather than reject. A merge action can still fail the binary acceptance rule if it includes a blocking issue, so action percentages need not equal RAP. The distribution alone does not establish whether concerns are excessive or justified.

RQ3 (cited concerns). We count reviewer-reported concerns without adjudicating their truth. The naive prompt often cites insufficient tests (F7), consistent with the omission of held-out tests from packets; the data do not identify that omission as the sole cause. After the joint adjustment, F7 is rare in Qwen-14B's non-accepted agent verdicts (1 Lite patch, 0 Verified patches). Regression risk (F2), fragility (F8), and convention mismatch (F6) remain (Table V). Their factual validity and practical importance require separate adjudication.

Table VI distinguishes gold rejection from gold blocking. On Lite, Qwen-14B rejects $26/96$ gold patches but reports blocking issues on only $3/96$. Across the adjusted Qwen models and splits, gold blocking is 3–8%. A non-accepting decision can therefore occur without a blocking label. These counts concern schema outputs, not whether the underlying rationale is fabricated or a maintainer considered it.

RQ4 (reviewer agreement). Fleiss' κ for the three adjusted

TABLE VI
ADJUSTED QWEN VERDICTS ON GOLD. GOLD BLOCKING IS THE SHARE REPORTING AT LEAST ONE BLOCKING ISSUE, NOT AN ADJUDICATED HALLUCINATION RATE OR THE COMPLEMENT OF GOLD ACCEPTANCE.

Reviewer	Split	n_g	Gold acc.%	Gold blocking%	Gold rej.%
Qwen-7B	Lite	96	87.50	6.25	12.50
Qwen-7B	Verified	194	87.11	8.25	12.89
Qwen-14B	Lite	96	72.92	3.12	27.08
Qwen-14B	Verified	194	74.74	4.64	25.26
Qwen-32B	Lite	96	56.25	4.17	43.75
Qwen-32B	Verified	194	61.86	5.67	38.14

TABLE VII
INTER-REVIEWER AGREEMENT (FLEISS κ) ON COMMON VALID PACKETS. THE FIVE-REVIEWER PANEL INCLUDES QWEN, LLAMA, AND DEEPSEEK; a/g DENOTE AGENT/GOLD.

Panel / Split	n packets	κ_{all}	κ_a	κ_g
Qwen / Lite	192	0.1008	0.1327	0.0654
Qwen / Verified	388	0.1172	0.1222	0.1120
Five / Verified	383	0.0245	0.0471	0.0018

Qwen models is 0.1008 on all Lite packets and 0.1172 on Verified. Adding Llama and DeepSeek gives approximately 0.02 on common valid Verified packets (Table VII). Thus model selection changes individual review decisions as well as aggregate rates. A low κ is chance-corrected disagreement in the tested panel; it neither identifies which reviewer is correct nor validates the benchmark's historical reference as an oracle.

RQ5 (coding-agent scaffolds). We compare the adjusted Qwen panel on RAG+GPT-4 (14 resolved tasks), SWE-agent+Claude-3.5 (168), Agentless+GPT-4o (194), and OpenHands-CodeAct (265), each against gold on the same tasks (Table VIII). The 2.8–53.0% figures are official execution-based %Resolved, not reviewer scores. Qwen-14B does not detect a paired difference for any scaffold ($p =$ 1.0, 0.5601, 0.4996, and 0.3891). This statement must be reviewer-qualified: Qwen-32B accepts only 35.1% of SWE-agent patches versus 62.5% of paired gold, and 29.8% of OpenHands patches versus 58.5% of paired gold, with $p <$ 0.001 in both cases. The null does not hold uniformly across the adjusted panel. Qwen-7B comparisons are non-significant, but this does not establish equivalence. RAG's 14 tasks and only two discordant pairs for Qwen-14B provide little power, so we draw no evidence of equivalence from that result. None of these tests establishes how maintainers would rank the scaffolds' patches.

C. Adapted binary-judge pilots

Following the preprints (Shi: arXiv:2511.10865v1), we adapt Cihan's classification task [11] and Shi's *unrefined draft-rubric ablation* [12], alongside a test-blind maintainer comparator. All use the same pinned Qwen2.5-Coder-7B-Instruct, greedy decoding, full issue and candidate diff, and binary JSON output; execution labels and candidate source are hidden. Shi alone additionally derives a per-bug rubric from the developer patch. Thus information access differs; this is not a reproduction of the full original systems.

Each pilot selects ten officially resolved and ten unresolved Agentless Verified tasks by fixed hash order, subject to recorded length limits, and pairs agent/developer patches. The first 120 judgments produced 37 Markdown-fenced JSON format failures under the frozen strict parser; fence-only recovery is a separate post-hoc analysis. We then freeze a fence-tolerant parser and a disjoint 20-task follow-up before collecting its 120 judgments (all valid), without changing judging prompts. Table IX reports only this follow-up. Across both pilots there are 240 judgments and 40 rubric drafts.

Agreement's Wilson 95% intervals are 48.1–85.5%, 43.3–81.9%, and 53.1–88.8%, respectively. These small samples support no superiority claim. Inherited execution labels are not new test runs or mergeability truth. Cihan's function task is adapted to diffs without correction generation; Shi's two-human refinement and human-consensus labels remain absent. Additional reference information also biases its developer-patch acceptance. These pilots do not repair the archived negative inputs or validate the historical reviewers.

A separately frozen extension evaluates all Lite (300) and Verified (500) gold patches using the same test-blind maintainer adapter, without agent-based selection or truncation. The 93 shared tasks are evaluated once (707 unique gold), alongside all 302 available unresolved candidates; all $1,009$ calls complete, with three malformed-JSON responses. Gold acceptance is $143/299$ valid on Lite and $284/499$ on Verified (one failure per split). Among 300 valid same-task failing/developer pairs, acceptance is $135/300$ versus $149/300$; discordant counts are $(47, 61)$ (exact McNemar $p = .211$). Four empty candidates and two incomplete pairs are excluded explicitly. This is not equivalence, new harness execution, or independent calibration.

D. Qualitative inspection of cited concerns

A post-hoc technical audit selects the first two nonaccepted hosted-reviewer cases in lexical order within each of three strata: resolved agent (30 available), gold (3), and failing agent (38). Archived diffs are applied to pinned upstream files; extracted functions/branches run with synthetic inputs and backend stand-ins. Five focused claims have local support; the gold `django-11179` regular-path stale-PK objection is contradicted by existing cleanup. These checks are not independent adjudication or an error-rate estimate. The `django-11555` phrase "the one Django merged" is a recognition cue, not proof of training exposure.

For all three hosted nonaccepted gold cases, a separate 365-day default-branch audit finds 12, 93, and 35 source-file commits (Django, Matplotlib, scikit-learn). File churn is not defect count. Matplotlib issue #27103 explicitly links the original PR commit; fix #27179 restores the default minimum to one and adds an empty-maximum regression test, supporting the empty-bin concern. No matching regression was identified for the other two concerns in this scoped inspection. This is not absence of defects; other branches, renames and indirect

TABLE VIII

ACROSS FOUR AGENTS SPANNING 2.8–53% %RESOLVED ON VERIFIED, ADJUSTED QWEN RESULTS VARY BY REVIEWER. RAP IS AGENT ACCEPTANCE; EACH PAIRED p COMPARES AGENT AND GOLD ON THE SAME TASKS. QWEN-32B DETECTS DIFFERENCES FOR SWE-AGENT AND OPENHANDS. RAG HAS LOW POWER.

Coding agent	n	%Resolved	RAP_7	RAP_{14}	RAP_{32}	p_7	p_{14}	p_{32}
RAG+GPT-4	14	2.8	92.9	78.6	50.0	1.0000	1.0000	0.2500
SWE-agent+Claude-3.5	168	33.6	92.3	78.6	35.1	0.5235	0.5601	< 0.001
Agentless+GPT-4o	194	38.8	91.2	72.2	57.7	0.1516	0.4996	0.3497
OpenHands-CodeAct	265	53.0	89.4	79.2	29.8	0.7660	0.3891	< 0.001

TABLE IX

DISJOINT FOLLOW-UP PILOT: AGREEMENT WITH INHERITED FUNCTIONAL LABELS ON AGENT PATCHES (A), REJECTION OF UNRESOLVED PATCHES (N), AND DEVELOPER-PATCH ACCEPTANCE (G). THESE ARE ADAPTED METHODS, NOT MAINTAINER-VALIDATION OR SUPERIORITY RESULTS.

Adapted judge	A ($n = 20$)	N ($n = 10$)	G ($n = 20$)
Maintainer	$14/20$	$7/10$	$9/20$
Cihan	$13/20$	$7/10$	$8/20$
Shi draft	$15/20$	$6/10$	$11/20$

effects are unmeasured. The public follow-up predates the 2026 judging and may itself have been recognized.

E. A human-review pilot

Author Myeongmuk Kang, a non-independent, source-blinded rater, reviewed 40 packets from 20 Agentless Verified paired tasks. The sampling script sorts task IDs, shuffles with `Random(7)`, selects 20, and then shuffles the 40 agent/gold packets. The form uses the adjusted rubric, but limits issues to $1{,}800$ characters and diffs to $9{,}000$, versus $6{,}000$ and $24{,}000$ in the LLM path. Thus shared rubric does not mean identical input context. The archive does not document Kang's prior code-review experience or precise role in rubric development; these remain limitations rather than inferred credentials.

This rater accepts $15/20$ agent and $13/20$ gold packets (75% and 65%), with paired $p = 0.75$. Agreement with adjusted open reviewers ranges approximately from -0.22 to 0.07, and with the archived hosted reviewer is $\kappa = 0.167$. This sample provides descriptive observations only. It does not establish what humans in general would accept, whether context truncation caused disagreement, or that any LLM or human escapes an operational dilemma. Independent raters and context-controlled comparisons are needed for such claims.

V. DISCUSSION

A. Interpretation and diagnostic guidance

An acceptance rate needs a reference. Gold outcomes offer a known historical decision against which a reviewer can be compared cheaply, but not a defect-free truth label. A worked example shows why the reference matters: on Lite the naive Qwen-14B accepts $6/96$ agent patches and $5/96$ gold patches. The adjusted configuration accepts $68/96$ and $70/96$, respectively. Reading only the agent column would suggest a large change in patch quality even though the candidate patches are unchanged. The comparison instead shows sensitivity to the assembled review configuration. It does not isolate which prompt change caused that sensitivity. The diagnostic decision is to withhold a mergeability interpretation for both configurations: historical-gold disagreement limits the naive judge, while reused development tasks and the flawed failing-patch condition leave the adjusted judge unvalidated. Higher acceptance alone is insufficient.

Separate three questions. Historical-gold agreement, sensitivity to functional failure, and agreement with present-day maintainers are different outcomes. The paired hosted result answers a reviewer-specific agent/gold question, and the pilot reports one rater's judgments. The archived failing-patch rates answer neither functional-discrimination under correct test information nor a causal test-status question. Consequently, neither the hosted reviewer nor an open model is validated for real merge-decision support.

A diagnostic protocol. Future benchmark studies can (1) document source-blinded packet contents and truthful test-status information, (2) report historical-gold acceptance alongside agent acceptance, (3) develop rubrics using tasks separate from evaluation, (4) introduce appropriate failing-patch controls with explicit task selection and accurate instructions, (5) report valid counts, paired differences, missingness, and multiple-reviewer agreement, and (6) obtain independent maintainer validation before making practical merge recommendations. This is benchmark-design guidance, not a validated decision aid or a universal numeric pass/fail rule. Our study demonstrates the need for these distinctions; it does not claim to have completed every step.

Scope of the observed tension. The correlation across open configurations and low agreement show dependence on judge choice in this sample. They do not identify a single leniency mechanism or prove all open models incapable of discrimination. Qwen-32B's cross-scaffold differences and the hosted operating point make a blanket agent–gold null inaccurate. Model scale alone also does not explain the Qwen observations; different checkpoints may have different training and alignment histories.

Analysis reproduction versus fresh inference. Stored labels and verdicts regenerate tables and figures without model calls. New reviews require a serving environment; exact hosted reproduction remains unresolved because its snapshot is unrecorded. Templates, keys, verdicts, and regeneration code support computational checking, not adjudication or recovery

of missing identity.

B. Threats to validity

Construct. Historical merge outcomes can tolerate imperfections and depend on PR discussion, CI, project plans, and social context absent from packets. Our binary labels operationalize a reviewer decision, not a verified maintainer decision or an objection's truth. The technical probes and three-case history audit provide no independent rationale adjudication or population defect incidence. Historical merges can contain defects. A single non-independent rater cannot supply that validation. Its shorter input limits further complicate human–LLM comparisons.

Internal. Rubric development reused the evaluation gold cohort, and intermediate versions, inspected IDs, and iteration count were not preserved. Five subsets of fixed verdicts do not establish task independence. The two prompt changes were not separated. Negative-control inputs incorrectly asserted a pass and used disjoint task populations; therefore their apparent separation cannot validate discrimination in the intended condition. The hosted 81-packet execution sample also has an unexplained missing position.

Contamination and blinding. Agents and reviewers may recognize public issues, repository states or fixes. Explicit source masking cannot remove pretraining exposure; either gold or agent acceptance could be affected, with unmeasured direction/magnitude. Equal aggregate rates do not rule out recognition. This differs from documented gold-task reuse in rubric development; public post-merge histories also predate the judging.

External. Selected Python tasks, four coding-agent prediction sets, five open-weight reviewers and one unrecorded hosted snapshot do not cover all methods, languages or model versions. Historical positive controls use resolved subsets; the full-split extension covers only the test-blind 7B adapter. RAG's 14 tasks support no broad weak-agent claim. Industrial effort and deployment effects are unmeasured.

Statistical and computational. Tests are exploratory and use valid complete pairs. Historical parsing, normalization and higher-temperature retries can affect the population; their effects were not isolated. Wilson intervals rely on binomial assumptions; aggregate correlations are descriptive. Non-significance is not equivalence, and low agreement identifies no correct rater. Serving/numerical dependencies and the hosted identity gap limit exact inference reproduction.

VI. Related Work

Execution and patch correctness. Plausible but incorrect repairs are a longstanding APR concern [4], [5]. GenProg, Defects4J, and language-model repair [19]–[22] motivate checking beyond available tests. Patch assessment uses behavioral or learned evidence [6], [7]; our outcome is acceptance of repository review packets relative to historical merge decisions.

Human-guided assessment. Shi et al. [12] generate per-bug rubrics, obtain human review/refinement, and judge patch validity against human consensus. Cihan et al. [11] compare judgments of generated HumanEval code and canonical solutions. Shared rubrics and positive controls are precedents, not our novelty. Historical repository merges are not stronger truth than adjudicated human labels. Our non-independent human pilot is not comparable validation; the additional method adaptations in §IV-C likewise omit Shi's human-refinement stage.

Review automation. Industrial review includes design, conventions, and knowledge transfer [2], [23], [24]. Tufano et al. [3], prompting/fine-tuning comparisons [25], LLaMA-Reviewer [26], and Atlassian's context-aware RovoDev [27] study review activities, comments, or revisions. These objectives differ from our binary historical-acceptance comparison; we do not reproduce all these systems or claim their deployment outcomes.

Judge reliability. LLM judges [8] exhibit comparison and self-preference biases [28], [29]. Our contribution is a SWE-bench-specific diagnosis using historical gold, cross-agent paired decisions, and reviewer agreement, with calibration reuse and flawed archived negative inputs explicitly limiting the claims.

VII. Conclusion

Review-based scores require examining the judge as well as the patch. Historical prompt adjustment changes agent and gold acceptance; reviewer/scaffold differences and paired results do not establish equivalence or maintainer validity. The hosted identity is missing and archived negative inputs are flawed. New full-split and matched controls remain limited to a test-blind 7B adapter; targeted code/history checks establish no population objection-error rate. Independent calibration and human validation remain necessary before operational merge decisions.

Data and Artifact Availability

Version `2026.11.05-v8` at https://github.com/myoungmuk/icse2027-6658-artifact-20261105-v2 contains preserved inputs/outputs and regeneration instructions for historical analyses, adapted pilots, full-split controls and technical/history checks. Analysis reproduction does not recover the missing hosted identity or rubric history, or rerun inference.

Acknowledgments

OpenAI Codex [30] assisted revision, analysis, figures, reference checks and implementation. The authors are responsible for all claims and the final manuscript.

REFERENCES

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024, pp. 54 107–54 157. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[2] C. Sadowski, E. Söderberg, L. Church, M. Sipko, and A. Bacchelli, "Modern code review: A case study at google," in *International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2018, pp. 181–190. [Online]. Available: https://doi.org/10.1145/3183519.3183525

[3] R. Tufano, L. Pascarella, M. Tufano, D. Poshyvanyk, and G. Bavota, "Towards automating code review activities," in *International Conference on Software Engineering (ICSE)*, 2021, pp. 163–174. [Online]. Available: https://doi.org/10.1109/icse43902.2021.00027

[4] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Joint Meeting on Foundations of Software Engineering (FSE)*, 2015, pp. 532–543. [Online]. Available: https://doi.org/10.1145/2786805.2786825

[5] Y. Qi, X. Mao, Y. Lei, Z. Dai, and C. Wang, "The strength of random search on automated program repair," in *International Conference on Software Engineering (ICSE)*, 2014, pp. 254–265. [Online]. Available: https://doi.org/10.1145/2568225.2568254

[6] S. Wang, M. Wen, B. Lin, H. Wu, Y. Qin, D. Zou, X. Mao, and H. Jin, "Automated patch correctness assessment: How far are we?" in *International Conference on Automated Software Engineering (ASE)*, 2020, pp. 968–980. [Online]. Available: https://doi.org/10.1145/3324884.3416590

[7] Y. Xiong, X. Liu, M. Zeng, L. Zhang, and G. Huang, "Identifying patch correctness in test-based program repair," in *International Conference on Software Engineering (ICSE)*, 2018, pp. 789–799. [Online]. Available: https://doi.org/10.1145/3180155.3180182

[8] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-bench and chatbot arena," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 46 595–46 623. [Online]. Available: https://doi.org/10.52202/075280-2020

[9] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-Based Software Engineering Agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025. [Online]. Available: https://doi.org/10.1145/3715754

[10] OpenAI, "Introducing SWE-bench verified," https://openai.com/index/introducing-swe-bench-verified/, 2024.

[11] U. Cihan, A. İçöz, V. Haratian, and E. Tüzün, "Evaluating Large Language Models for Code Review," arXiv:2505.20206, 2025. [Online]. Available: https://arxiv.org/abs/2505.20206

[12] S. Shi, R. Wei, M. Tufano, J. Cambronero, R. Cheng, F. Ivančić, and P. Rondon, "Towards a Human-in-the-Loop Framework for Reliable Patch Evaluation Using an LLM-as-a-Judge," in *Proc. 1st Int. Workshop on AI for Software Quality Evaluation (AI-SQE)*, 2026, pp. 19–28. [Online]. Available: https://doi.org/10.1145/3786175.3788348

[13] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 50 528–50 652. [Online]. Available: https://doi.org/10.52202/079017-1601

[14] X. Wang, B. Li, Y. Song, F. F. Xu *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *International Conference on Learning Representations (ICLR)*, 2025, pp. 65 882–65 919. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html

[15] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 1592–1604. [Online]. Available: https://doi.org/10.1145/3650212.3680384

[16] J. Yang, C. E. Jimenez, A. L. Zhang, K. Lieret *et al.*, "SWE-bench multimodal: Do AI systems generalize to visual software domains?" in *International Conference on Learning Representations (ICLR)*, 2025, pp. 2794–2829. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/07d6332ae36730707fddddba736d7b6c-Abstract-Conference.html

[17] M. Riddell, A. Ni, and A. Cohan, "Quantifying contamination in evaluating code generation capabilities of language models," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 14 116–14 137. [Online]. Available: https://doi.org/10.18653/v1/2024.acl-long.761

[18] B. Hui *et al.*, "Qwen2.5-coder technical report," arXiv:2409.12186, 2024. [Online]. Available: https://arxiv.org/abs/2409.12186

[19] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "GenProg: A generic method for automatic software repair," *IEEE Transactions on Software Engineering (TSE)*, vol. 38, no. 1, pp. 54–72, 2012. [Online]. Available: https://doi.org/10.1109/TSE.2011.104

[20] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *International Symposium on Software Testing and Analysis (ISSTA)*, 2014, pp. 437–440. [Online]. Available: https://doi.org/10.1145/2610384.2628055

[21] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2022, pp. 959–971. [Online]. Available: https://doi.org/10.1145/3540250.3549101

[22] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *International Conference on Software Engineering (ICSE)*, 2023, pp. 1430–1442. [Online]. Available: https://doi.org/10.1109/ICSE48619.2023.00125

[23] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in *International Conference on Software Engineering (ICSE)*, 2013, pp. 712–721. [Online]. Available: https://doi.org/10.1109/icse.2013.6606617

[24] P. C. Rigby and C. Bird, "Convergent contemporary software peer review practices," in *Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2013, pp. 202–212. [Online]. Available: https://doi.org/10.1145/2491411.2491444

[25] C. Pornprasit and C. Tantithamthavorn, "Fine-Tuning and Prompt Engineering for Large Language Models-based Code Review Automation," *Information and Software Technology*, vol. 175, p. 107523, 2024. [Online]. Available: https://doi.org/10.1016/j.infsof.2024.107523

[26] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, "LLaMA-Reviewer: Advancing Code Review Automation with Large Language Models through Parameter-Efficient Fine-Tuning," in *IEEE Int. Symp. Software Reliability Engineering (ISSRE)*, 2023, pp. 647–658. [Online]. Available: https://doi.org/10.1109/issre59848.2023.00026

[27] K. Tantithamthavorn, Y. Zou, A. Wong, M. Gupta, Z. Wang, M. Buller, R. Jiang, M. Watson, M. Jeong, K. Chen, and M. Wu, "RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-based Code Review Automation at Atlassian," in *Proc. IEEE/ACM Int. Conf. Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2026, pp. 34–45. [Online]. Available: https://doi.org/10.1145/3786583.3786851

[28] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, L. Kong, Q. Liu, T. Liu, and Z. Sui, "Large language models are not fair evaluators," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, pp. 9440–9450. [Online]. Available: https://doi.org/10.18653/v1/2024.acl-long.511

[29] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 68 772–68 802. [Online]. Available: https://doi.org/10.52202/079017-2197

[30] OpenAI, "Codex," Software tool, 2026, accessed November 3, 2026. [Online]. Available: https://openai.com/codex/